Faculty of Information Technology
MSc in Big Data Analytics

Big Data Analytics

Final Project

Distributed Multi-Model Analytics for E-commerce Data.

Done by:

MUTONI Denyse 101207

Code repository: github.com/denyse101207-bigdata/ecommerce-big-data-analytics

November, 2026

1. Summary

This project builds a multi-model analytics system for a synthetic e-commerce dataset: 5,000 users, 2,000 products in 25 categories, 100,000 browsing sessions and 25,000 transactions over 91 days (about $26.3 million of revenue). MongoDB stores the entity-centric documents, HBase stores time-series session events, and Apache Spark cleans the data, computes recommendation indicators, cohorts and SQL analytics, and joins data from both stores in an integrated Customer Lifetime Value (CLV) analysis. Seven charts and the main business findings are in Section 5.

1.1. System architecture and technology selection

The synthetic data produced by the provided generator is loaded into MongoDB and HBase, and read directly by Spark (Figure 1). Because HBase and Spark do not fit in the 8 GB of RAM of the development laptop at the same time, the stages run in sequence and the integrated analysis uses a staging step: each store exports the data it owns to JSON Lines files, and Spark joins them.

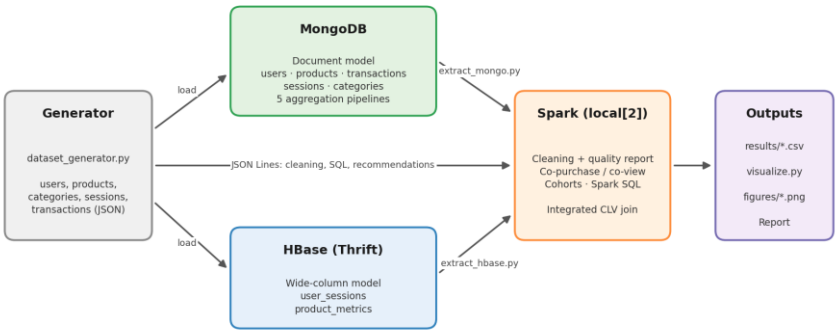


Figure 1. System architecture: MongoDB, HBase and Spark around the generated dataset.

Environment. Windows 11, 8 GB RAM; Python 3.14; Java 17; PySpark 4.3 in local mode (2 cores, 2 GB driver); MongoDB 8.0 as a Windows service; HBase 2.6.6 in standalone mode inside WSL2 (Ubuntu), accessed through its Thrift server with the happybase client.

Table 1. Technology selection by analytical task

Analytical task	Technology	Why this choice
Orders, users and catalog documents; revenue, popularity and segmentation queries	MongoDB	Nested data (order items, page views) is read and written as one unit; aggregation pipelines express the analytics without joins.
A user's latest sessions, time-range scans, daily product counters	HBase	Rows sorted by key give fast prefix and range scans; page views are sparse columns that cost nothing when absent; writes scale horizontally.
Cleaning, large joins, co-occurrence, cohorts, cross-source analysis	Apache Spark	Distributed DataFrames and SQL; the same code runs on a laptop or a cluster; can combine data exported from both stores.
Charts	Python / matplotlib	Reproducible static figures built from the saved result files.

2. Data modelling (Part 1)

2.1. MongoDB: document model

The design follows one rule: **embed what is read together, reference what is large or changes independently**. Table 2 shows each collection and the reason for its structure. All ISO date strings were converted to real dates, and loading all five collections takes 12 seconds.

Table 2. MongoDB collections

Collection	Document design	Reason
categories (25)	Subcategories embedded as an array with profit_margin	Small reference data that is always read together.
products (2,000)	category {id, name} and subcategory {id, name, profit_margin} embedded; price_history array	A product page needs all of it in one read; category names rarely change.
transactions (25,000)	items array embedded; each item enriched with category_id, subcategory_id and profit_margin	An order is always handled as a unit; the copied fields let revenue and profit by category run without joining products.
users (5,000)	geo_data, dates, plus embedded purchase_summary (orders, total spent, average order value, first/last purchase, top category) and engagement (sessions, duration, converted sessions)	Summaries are computed once at load time, so segmentation reads a single collection.
sessions (100,000)	page_views array embedded; cart_contents converted from a dictionary keyed by product_id into a cart_items array	The whole journey is one document; dynamic keys cannot be indexed or aggregated, an array can.

Seven indexes match the query patterns: transactions on (user_id, timestamp), timestamp and items.product_id; sessions on (user_id, start_time) and viewed_products; products on category.category_id; users on purchase_summary.total_orders.

Aggregation pipelines. Five pipelines were implemented: top products by revenue, revenue and estimated profit by category, daily revenue, user segmentation by purchase frequency, and most-viewed products. The segmentation pipeline uses $bucket on the embedded summary and touches one collection; the category pipeline needs only a lookup of the 25 category names (shown in mongo-shell form; the project runs the same pipeline from Python):

```
db.transactions.aggregate([
  {$unwind: "$items"},
  {$group: {_id: "$items.category_id", revenue: {$sum: "$items.subtotal"},
          est_profit: {$sum: {$multiply: ["$items.subtotal", "$items.profit_margin"]}}}},
  {$lookup: {from: "categories", localField: "_id", foreignField: "_id", as: "c"}},
  {$sort: {revenue: -1}} ])
```

2.2. HBase: wide-column model

Table 3. HBase tables

Table	Row key	Column families	Purpose
user_sessions	user_id#reverse_timestamp (9999999999999 minus start time in ms)	s: session metadata; pv: one sparse column per page view (pv:00, pv:01, …); cart: one column per product (cart:product_id)	A user's latest sessions and time-range scans

Table	Row key	Column families	Purpose
product_metrics	product_id#yyyymmdd	m: views, cart_adds, purchases, units_sold, revenue	Daily product performance and funnel rates

The reverse timestamp makes a user's sessions adjacent **and newest first**, so "the last N sessions of user X" is a short prefix scan and a date range is a single range scan, with no sorting. Sessions have 4 to 16 page views, so rows have different numbers of columns, which a wide-column store handles at no cost. One version per cell is kept because events are not updated.

```
create 'user_sessions', {NAME=>'s', VERSIONS=>1}, {NAME=>'pv', VERSIONS=>1},
{NAME=>'cart', VERSIONS=>1}
create 'product_metrics', {NAME=>'m', VERSIONS=>1}
scan 'user_sessions', {STARTROW=>'user_000000#', STOPROW=>'user_000000$', LIMIT=>5}
scan 'product_metrics', {STARTROW=>'prod_01434#20260801',
STOPROW=>'prod_01434#20261101'}
```

Loaded data and queries. A random sample of 30,000 sessions was loaded into user_sessions, and 151,988 daily product rows were built from all sessions and transactions (48 seconds in total). The queries returned a user's five latest sessions in newest-first order, the 7 sessions of the same user between 1 September and 31 October, and a full session as a single row (page views and cart). For product prod_01434 the period totals were 236 views, 72 cart additions and 32 purchases, a 30.5% view-to-cart and 13.6% view-to-purchase rate (purchases include orders without a session, so the second rate is overstated).

2.3. Why each kind of data sits where it does

- **Users, products, categories, transactions: MongoDB only.** They are entities with relationships and need flexible filtering and aggregation; HBase offers only row-key access, no secondary indexes and no joins.
- **Sessions: both.** MongoDB holds all 100,000 as documents for analytics such as most-viewed products. HBase holds a sample keyed for the access pattern it is best at: the latest activity of one user and time-range scans.
- **Daily product metrics: HBase only.** Append-heavy time series with one row per product and day, read by range.

3. Data processing with Apache Spark (Part 2)

3.1. Pipeline, cleaning and data quality

The batch pipeline has five steps: (1) convert the JSON arrays to JSON Lines so files can be split across tasks; (2) read every dataset with an explicit schema; (3) clean and normalize: parse timestamps, drop duplicates by key, lower-case categorical fields, fill missing values and validate business rules; (4) reconcile derived fields, such as each user's last activity; (5) analyse and save small result tables. Table 4 summarizes the data-quality report.

Table 4. Data-quality findings (Spark)

Check	Result	Note
Rows removed by cleaning (users, products, sessions, transactions)	0	No duplicates, nulls or invalid values

Check	Result	Note
Broken references (sessions→users, orders→users, items→products, orders→sessions)	0	Referential integrity holds
Orders where total ≠ subtotal − discount	0	Totals are consistent
Users whose last_active is earlier than their real activity	4,888 (97.8%)	Recomputed from last session and last order
Orders without a session	12,285 (49.1%)	Cannot be traced to browsing
Sessions labelled "converted" with no order	7,697	See Section 5 for the likely cause
Inactive products / products with zero stock	89 / 98	Informational

3.2. Recommendation indicators and cohort analysis

Co-purchase ("users who bought X also bought Y") uses a transaction as the basket; **co-view** uses a session as the basket (85,959 sessions with 2 to 30 products viewed). For each product pair the job computes the pair count, support, confidence and lift = P(A,B) / (P(A)·P(B)); pairs seen fewer than 2 (purchase) or 3 (view) times are dropped as noise. The strongest purchase pair occurs in only 3 of 25,000 orders (lift up to 142) and the strongest view pairs in 5 to 6 sessions (lift about 14 to 15): the method works, but the random data contains almost no real buying pattern.

Cohort analysis. Users are grouped by registration month (February to August 2026); for each cohort and month since registration the job reports active buyers, buyer rate and revenue per user. In every complete month 76% to 84% of a cohort buys, with $1,533 to $2,006 of revenue per user, and no cohort decays. The very low values in each cohort's last row (6% to 9%) are partial months at the end of the data and not churn.

3.3. Spark SQL and optimization choices

Six Spark SQL queries run on views of the cleaned data: top three categories per month (window function RANK), customer spend quartiles (NTILE), conversion funnels by device and by referrer, a 7-day moving average of daily revenue, and a "user 360" join of profile, sessions and orders that stands for data from MongoDB and HBase. A simplified version of the cohort query:

```
WITH cohort AS (SELECT user_id, date_trunc('month', registration_date) AS cohort_month
FROM users),
activity AS (SELECT c.cohort_month, t.user_id, t.total,
    CAST(months_between(date_trunc('month', t.txn_ts), c.cohort_month) AS INT) AS
month_offset
  FROM transactions t JOIN cohort c ON t.user_id = c.user_id)
SELECT cohort_month, month_offset, COUNT(DISTINCT user_id) AS active_buyers,
SUM(total) AS revenue
FROM activity WHERE month_offset >= 0 GROUP BY cohort_month, month_offset
```

Optimization choices: JSON Lines instead of one large JSON array; explicit schemas (the cart is declared as a MAP, because schema inference would create a column for each of the 2,000 product ids); broadcast joins for the small product and category tables; built-in Spark functions only, with no Python UDFs; a basket-size cap of 30 products to stop the pair join from exploding; 8 shuffle partitions and 2 cores to suit the machine; and only small aggregated results collected to the driver. On the laptop the jobs ran in 115 s (cleaning and quality report), 74 s (recommendations and cohorts), 67 s (SQL) and 43 s (CLV).

4. Integrated analytics: Customer Lifetime Value (Part 3)

Business question. How much is each customer worth over the next 12 months, and do the most valuable customers also browse more?

Data sources and workflow.

1. MongoDB supplies the user profiles with their embedded purchase summary and the transactions, including estimated profit computed from the margins embedded in each item (extract_mongo.py).
2. HBase supplies the browsing sessions: number, duration and conversions per user, taken from the user_sessions rows (extract_hbase.py).
3. Spark joins both sources by user_id. Per user it computes orders, average order value (AOV), orders per month over the 91-day window, margin rate and days since the last order; then predicted 12-month revenue = AOV × orders per month × 12, and predicted profit = revenue × margin rate. Customers are split into quartiles of predicted profit (Platinum, Gold, Silver, Bronze).
4. Integration checks: all 5,000 users matched; 4,987 have HBase sessions; 4,970 have orders; and for 0 users the summary stored in MongoDB differs from the sum of their transactions.
5.

Table 5. CLV tiers (1,250 customers each)

Tier	Users	Spend so far	Predicted 12-m revenue	Predicted 12-m profit	Share of profit	Sessions (HBase)	Days since last order
Platinum	1,250	$8,908	$35,240	$9,403	43.5%	6.4	11.0
Gold	1,250	$5,871	$23,226	$6,010	27.8%	6.2	14.2
Silver	1,250	$4,193	$16,588	$4,183	19.4%	5.9	17.9
Bronze	1,250	$2,080	$8,229	$2,002	9.3%	5.5	26.2

Why these technologies. The profit estimate needs no join because MongoDB's order items carry their margin; HBase provides the engagement signal from its time-series rows; Spark performs the cross-source join and the window-based tiering. Spark connectors for MongoDB and HBase were not used, to keep the setup light and avoid matching connector and Spark versions on an 8 GB laptop; the export-and-join approach is simple and keeps each system doing what it does best.
Performance: the CLV job runs in 43 s on 5,000 users, 25,000 orders and 30,000 sessions; at larger scale the staging step would be replaced by connectors or HBase snapshots, and the join would be partitioned by user_id.

5. Visualizations and business insights (Part 4)

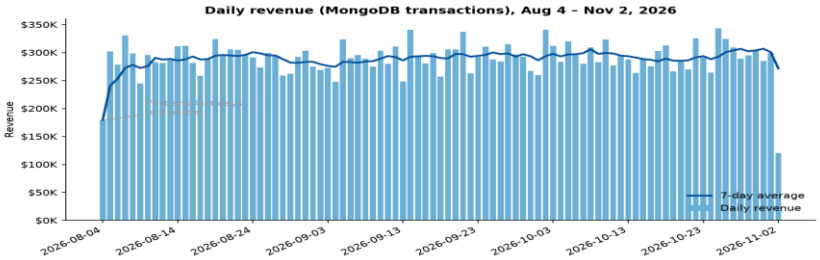


Figure 2. Daily revenue with 7-day average. Revenue is stable at about $290K per day ($26.3M in 91 days); the first and last days are partial.

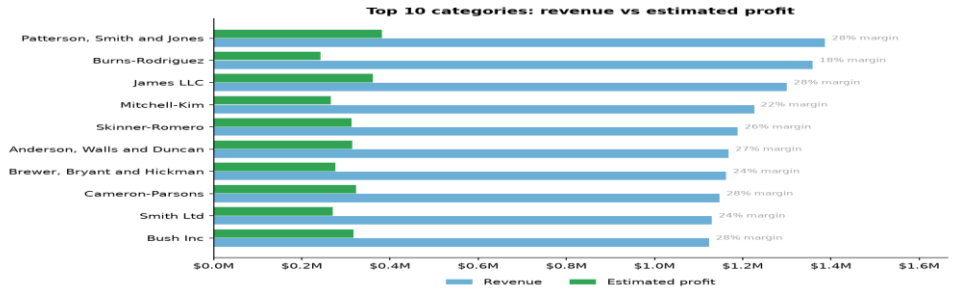


Figure 3. Revenue and estimated profit by category (top 10). Patterson, Smith and Jones leads on both ($1.39M revenue, $382K profit, 27.5% margin). Burns-Rodriguez is second on revenue ($1.36M) but has the lowest margin of the top ten (17.9%), so it earns only $243K. Promotions should consider margin and not only revenue."

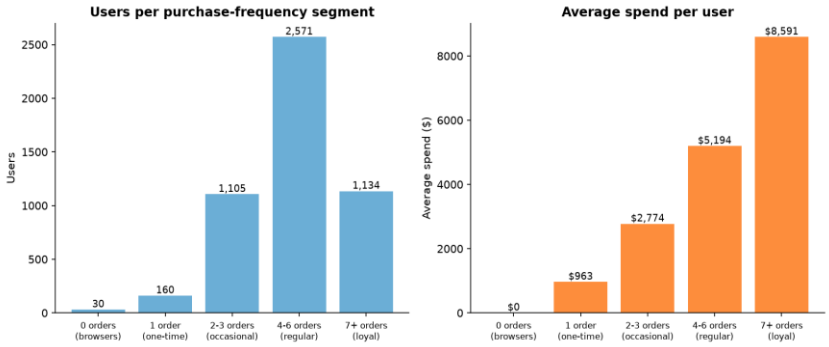


Figure 4. Customer segments by purchase frequency. The 4 to 6 order group is the largest (2,571 users, about 51% of revenue). The 7+ order group is 23% of users but 37% of revenue ($9.7M); average spend rises from $963 (one order) to $8,591 (7+ orders).

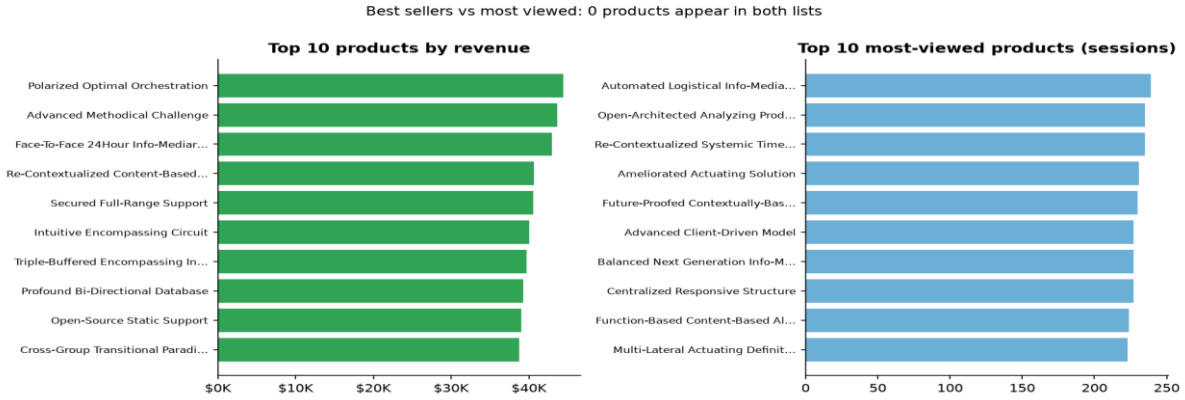


Figure 5. Top-selling versus most-viewed products. None of the ten best sellers is among the ten most viewed, and views are almost uniform (223 to 239 per product): in this dataset views do not drive sales. On real data this comparison finds products

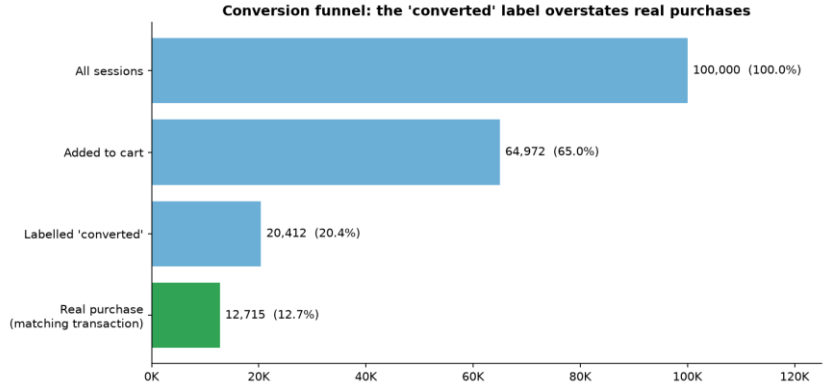


Figure 6. Conversion funnel. 65% of sessions add to cart, but only 12.7% match a real order (20.4% carry the "converted" label). The 7,697 labelled-converted sessions without an order are probably a generator artifact: it stops creating orders at its 25,000 cap, reached after about 60,000 to 70,000 sessions. True conversion therefore lies between 12.7% and 20.4%, and 69% to 80% of carts end without a purchase, which supports cart-recovery actions.

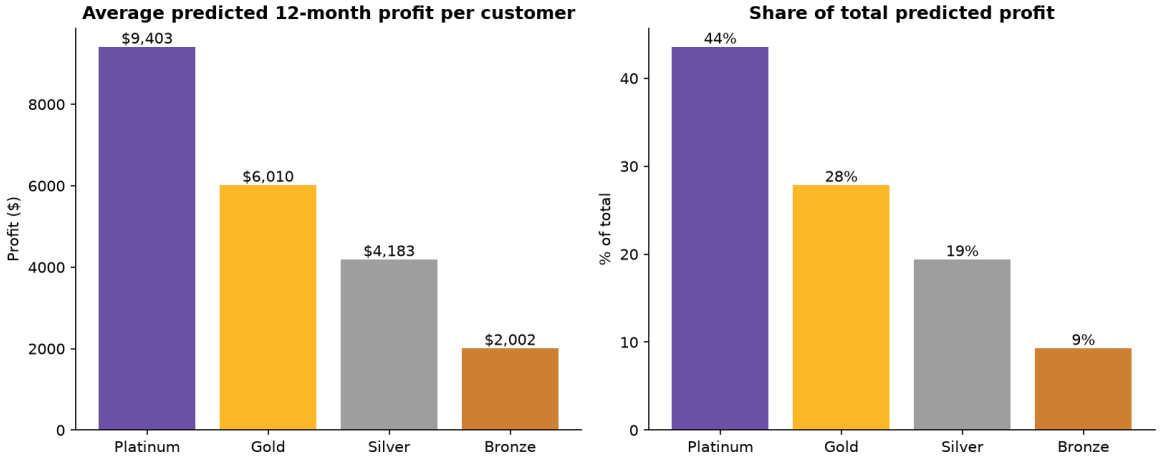


Figure 7. Predicted 12-month profit by CLV tier. A Platinum customer is worth 4.7 times a Bronze customer ($9,403 versus $2,002); the Platinum quarter of customers produces 44% of predicted profit and also bought more recently (11 versus 26 days ago). Retention offers belong with Platinum, and win-back messages with customers who have been silent for more than three weeks.

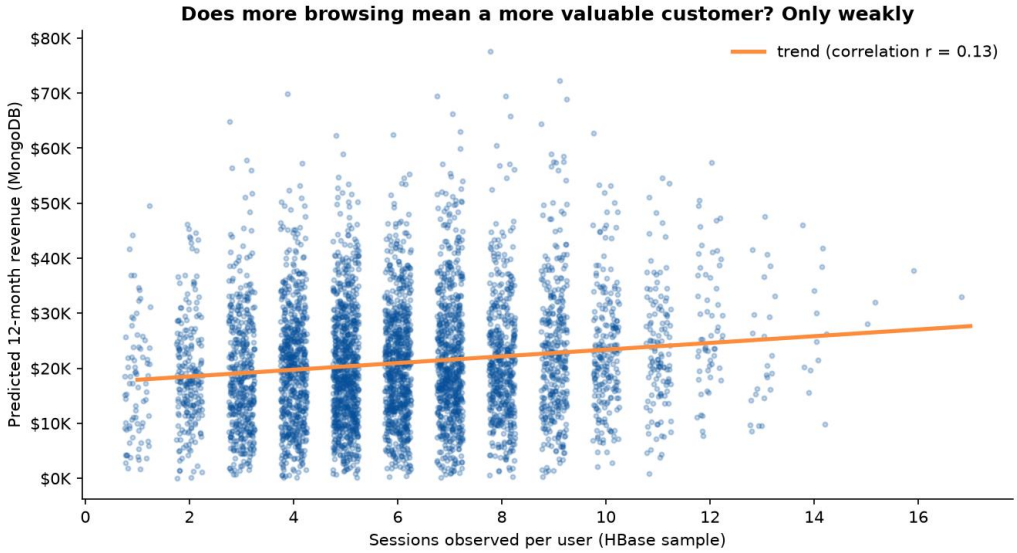


Figure 8. Browsing volume versus predicted value. The correlation is weak (r = 0.14 in Spark over all buyers; 0.13 in the chart, which excludes users without sessions), and session duration has none. Customers should be targeted by purchase behaviour and not by how much they browse. The 0.48 correlation between conversions recorded in HBase and orders recorded in MongoDB confirms that the two sources describe the same purchases.

6. Scalability considerations

- **MongoDB.** Shard sessions and transactions on a hashed user_id so a user's data stays together while load spreads out. Embedded arrays are bounded (at most 16 page views and 3 items), so documents stay far below the 16 MB limit; the user summaries can be kept current with change streams instead of reloads.
- **HBase.** The user_id prefix spreads writes across regions, and the reverse timestamp keeps recent data first; tables can be pre-split by user_id range, very active users salted if they become hot spots, and old sessions expired with a TTL.

- **Spark.** Move to a cluster manager (YARN or Kubernetes), store the cleaned data as partitioned Parquet instead of JSON, keep broadcast joins for small tables, and rely on adaptive query execution. The jobs already avoid collecting large data and cap basket sizes to limit skew in the pair join.

All timings reported here come from one 8 GB laptop and show that the pipeline is correct and efficient, not how it scales; the points above describe the design choices that allow scaling.

7. Limitations and future work

- **Synthetic data.** Device, referrer and product-pair effects are absent or weak by construction, cohorts do not decay, and best-selling products are unrelated to viewing. The findings show the methods, not real customer behaviour.
- **Missing demographics.** There is no age field, so segmentation uses purchase frequency and spend instead of age groups.
- **Generator effects.** The 25,000-order cap and the 49% of orders without a session make session-based funnels approximate; the corrected funnel gives a range and not a single rate.
- **Sampling and integration.** HBase holds a 30,000-session sample, and the cross-store join uses exported files and not live connectors.
- **CLV model.** It assumes each customer keeps their current order rate with no churn, over a short 91-day window.

Future work: probabilistic CLV models (BG/NBD with Gamma-Gamma); a Spark Structured Streaming simulation that writes session events into HBase in real time; MongoDB and HBase connectors for Spark; collaborative filtering (ALS) on real purchase data; and a container-based setup so the whole stack can be reproduced with one command.

Reproducibility. All code, result tables and charts are in the GitHub repository; the README lists the run order, from dataset_generator.py to visualize.py.